ARCHIVING AND COMPRESSION

ARCHIVING AND COMPRESSION TOGETHER

1.tar + gzip (z)

1. tar -czvf archive.tar.gz /etc
2. du -sh /etc
   6.5M
3. du -sh archive.tar.gz
   892K
4. tar -tvf archive.tar.gz
5. tar -xzvf archive.tar.gz -C /opt/

2.tar + bzip2 (j)

1. tar -cjvf archive.tar.bz2 /etc
2. du -sh /etc
   6.5M
3. du -sh archive.tar.bz2
   820K
4. tar -tvf archive.tar.bz2
5. tar -xjvf archive.tar.bz2 -C /opt/

3.tar + xz (J)

1. tar -cJvf archive.tar.xz /etc
2. du -sh /etc
   6.5M
3. du -sh archive.tar.xz
   668K
4. tar -tvf archive.tar.xz
5. tar -xJvf archive.tar.xz -C /opt/

CLOUDBLITZ

Note: -

1. c ----------> create archive
2. x ----------> extract archive
3. t ----------> list contents (list of content of a tar file)
4. f ----------> file name
5. v ----------> verbos (show process)
6. C ----------> change directory (extract path)
7. z ----------> gzip compression
8. j ----------> bzip2 compression
9. J ----------> xz compression

SCHEDULING

WHAT IS SCHEDULING

Scheduling simply means running the commands or tasks automatically at a specific time or interval.

Why Scheduling is Used

- Backup files automatically
- Run scripts daily/weekly
- System maintenance
- Logs cleanup

TYPES OF SCHEDULING

1.at -----------> non periodic task
2.cron --------> periodic task
3.anacron ---> desktop task

AT

it is a Linux command line utility used to schedule a job for a later execution

Example:

1.at "07:28" ----------> take time to schedule
> touch file
> mkdir dir

2.ctrl + d ----------> for save and quit
3.atq ----------> to check the schedule jobs
4.atrm (job number) ----------> to cancel the job
5.ls

CRON-JOBS

It is a Linux command line utility to schedule a job to run in the background automatically.

Criteria: < * * * * * command >

1.minute (0-59)
2.hour (0-23)
3.day of month (date)
4.month (1-12)
5.day of week (0-6) (sunday 0 or 7)

CLOUDBLITZ

COMMANDS

1./etc/crontab ----------> system wide configuration

2.crontab -e ----------> user specific configuration

(System-wide configuration)

Example: 1

1.Run at every minute

1. vim /etc/crontab

```
<min>  <hour>  <date>  <month>  <week>  <user>  <command>
  *        *        *         *         *       root     /usr/bin/touch   /root/file.txt
```

2. systemctl restart cron

3.cat /etc/crontab

```
  * * * * * root /usr/bin/touch /root/file.txt
```

4.ls

Example: 2 –

2.Run script at every 5 minute

1.vim script.sh

```
  #!/bin/bash
  echo "Cron job running at $(date)" >> /root/output.txt
```

2. chmod +x script.sh

2. vim /etc/crontab

```
<min>  <hour>  <date>  <month>  <week>  <user>  <command>
  */5      *        *         *         *       root     /root/script.sh
```

2. systemctl restart cron

3.cat /etc/crontab

```
  * * * * * root /root/script.sh
```

4.ls

Note:

1. This file (/etc/crontab) is generally used by root/admin
2. If you want to run as a local user, just replace root with your username

---

Example – 3 (user specific):

1.crontab -e

```
<min>  <hour>  <date>  <month>  <week>  <command>
  *        *        *         *         *       /usr/bin/touch   /home/mayur/file.txt
```

2.crontab -l

* * * * * /usr/bin/touch /home/mayur/file.txt

3.crontab -r

4.ls

Example – 3

1.crontab -e

<min> <hour> <date> <month> <week> <command>

* * * * touch /opt/demo

2.crontab -l

* * * * * touch /opt/demo

3.crontab -r

4.ls

Note:

1. Crontab -e ---------> to create a cron job
2. Crontab -l ---------> to list the cron jobs
3. Crontab -r ---------> to delete the cron jobs

Practice Examples:

1.10 sept 20:45

45 20 10 sept *

2.at 7:00am on Monday to Friday

0 7 * * 1-5

3.perform task only on Saturday

* * * * 6

4.every minute

* * * * *

5.every 5 minutes

*/5 * * * *

6.every 180 minutes

0 0,3,6,9,12,15,18,21 * * *

7.for every hour

0 * * * *

8.starts from 6:08pm and execute at 5 min intervals

CLOUDBLITZ

1. Syntax: 8-59/5 18-23 * * *
   {08, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58}
   So, it will run:
   - 6:08 PM
   - 6:13 PM
   - 6:18 PM
   - ...
   - up to 11:58 PM

9. 12 midnight daily

0 0 * * *

10. 1st of every month, 8:30 AM

30 8 1 * *
